

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY
PER LE TECNOLOGIE DELLA INFORMAZIONE E DELLA COMUNICAZIONE
DEL PIEMONTE

Sede legale: Torino, Piazza Carlo Felice 18

Oggetto:

PROVA TEORICO-PRATICA

**[D68444-10-2023-0] - Tecnico superiore per i metodi e le tecnologie per lo
sviluppo di sistemi software – WEB DEVELOPER**

Durata: 6 ore

Data: 08/07/2025

Ora consegna finale: 18:00

Cognome Nome studente: Speciale Gabriele

Firma studente: *Speciale Gabriele*

INTRODUZIONE

La seguente prova è composta da due fasi distinte:

- Un tema pratico progettuale che dovrà essere svolto secondo le modalità e le indicazioni fornite nel presente documento
- Due domande a risposta aperta su argomenti teorici a cui si dovrà rispondere tramite un testo scritto

Entrambe le due fasi sono parti integranti della prova ed entrambe saranno oggetto di valutazione. L'allievo/a per ottenere il punteggio massimo dovrà quindi svolgere entrambe le fasi.

La consegna degli elaborati verrà specificata nella sezione "al termine dell'elaborato" nel presente documento.

FASE PRATICA PROGETTUALE

OGGETTO

Web application per la gestione degli ordini in una pizzeria.

DESCRIZIONE

Gli allievi dovranno sviluppare un'applicazione web per la gestione delle prenotazioni al tavolo di una tipica pizzeria italiana. Deve essere possibile dichiarare il numero a tre cifre del tavolo e aggiungere una o più pizze all'ordine. Deve inoltre essere possibile consultare l'ordine effettuato.

La UI dovrà essere responsive e piacevole, studiata per essere visivamente accattivante e moderna.

BACKEND

Il backend è a disposizione ai seguenti URL:

<https://pizza-hub.synesthesia.dev>

<https://d1r0oonpv5yocu.cloudfront.net/>

REQUISITI

REQUISITI TECNICI

- Framework: ReactJS, Angular o NextJS (a scelta dello studente)
- Linguaggio: TypeScript
- UI: Responsive mobile first studiata per essere visivamente accattivante e giovanile

FUNZIONALITÀ RICHIESTE

Schermata iniziale:

- **Numero Tavolo:** L'utente deve inserire un numero di tavolo obbligatorio a tre cifre.
- **Pulsante "Inizia ad ordinare":** Quando premuto, viene visualizzata la pagina del menu contenente la lista delle pizze.

Schermata del Menu:

- **Lista delle Pizze:** La lista delle pizze deve essere ottenuta dall'API tramite la chiamata GET **/api/pizzas**.
- **Pulsante "Ordina":** Ogni pizza deve avere un pulsante "Ordina" che consenta all'utente di selezionare il numero di pizze da ordinare.
- **Selezione Quantità:** L'utente può selezionare una o più pizze dello stesso tipo da ordinare.
- **Prezzo Totale:** In basso deve essere visualizzato il numero delle pizze selezionate e il prezzo complessivo.
- **Pulsante "Conferma Ordine":** Al clic di questo pulsante, l'ordine viene inviato al tavolo tramite la chiamata POST **/api/tables/:table_number/orders**.

Schermata di Conferma Ordine:

- **Conferma Statica:** Dopo la conferma dell'ordine, l'utente vede una schermata di conferma statica.

Schermata di Elenco Ordine:

- **Lista delle Pizze ordinate:** L'utente può in qualunque momento successivo alla selezione del numero tavolo, visualizzare la lista delle pizze ordinate e il conto.

Il backend si occuperà automaticamente di completare l'ordine delle pizze ogni 15 minuti, simulando il processo di cucina.

DETTAGLIO SCHERMATE RICHIESTE

La **prima schermata** deve consentire di inserire un numero di tavolo obbligatoriamente a tre cifre. Una volta premuto sul pulsante **"Inizia ad ordinare"** viene visualizzata la **pagina di menu**, contenente la lista delle pizze. Il pulsante in alto a sinistra deve portare alla pagina iniziale, mentre il pulsante in alto a destra deve portare alla pagina il tuo ordine.

→ Aggancio API: nella schermata del menu è necessario chiamare l'API **GET /api/pizzas** per ottenere la lista delle pizze.

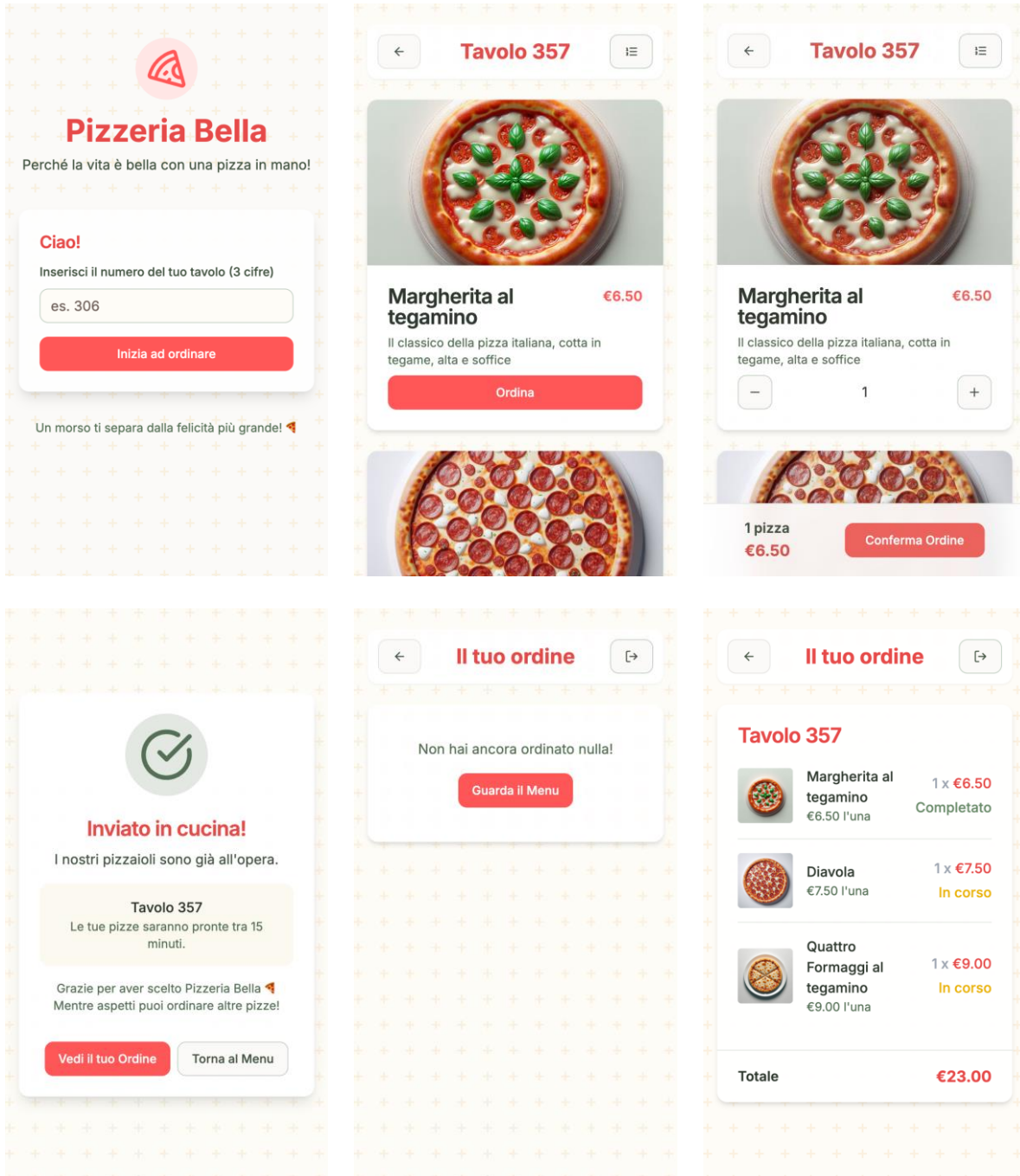
Premendo sul pulsante **"Ordina"** sotto alla pizza, deve essere possibile selezionare il numero di pizze dello stesso tipo da ordinare, una o più. In basso deve comparire il numero delle pizze selezionate, il prezzo complessivo e il pulsante **"Conferma Ordine"**.

→ Aggancio API: alla pressione del pulsante **"Conferma Ordine"** è necessario chiamare l'API **POST /api/tables/:table_number/orders** per inviare l'ordine al tavolo.

In seguito alla conferma dell'aggiunta all'ordine, deve essere visualizzata una schermata di conferma statica.

Si consiglia di utilizzare la rotta **/** per la pagina iniziale, la rotta **/ {table_number} /menu** per la pagina di menu, la rotta **/ {table_number} /confirmation** per la pagina di conferma ordine e la rotta **/ {table_number} /order** per visualizzare l'ordine.

N.b. La UI seguente è solo esemplificativa, l'allievo può proporre l'esperienza utente che desidera e reputa più adatta.



DOCUMENTAZIONE API

La documentazione in formato Swagger OpenAPI è disponibile all'endpoint `/api/docs`.

OTTENERE LISTA PIZZE

Endpoint	<code>/api/pizzas</code>
Metodo	GET
Descrizione	Restituisce la lista delle pizze disponibili.
Richiesta	Nessun parametro necessario
Risposte	200 OK: Ritorna la lista delle pizze.

OTTENERE DETTAGLIO PIZZA

Endpoint	<code>/api/pizzas/{id}</code>
Metodo	GET
Descrizione	Restituisce i dettagli di una pizza specifica.
Richiesta	Parametri: <code>id</code> (path): ID della pizza.
Risposte	200 OK: Dettagli della pizza. 400 Bad Request: ID della pizza non valido. 404 Not Found: Pizza non trovata.

OTTENERE LISTA ORDINI DEL TAVOLO

Endpoint	<code>/api/tables/{table_number}/orders</code>
Metodo	GET
Descrizione	Restituisce gli ordini associati a un tavolo.

Richiesta	Parametri: <code>table_number</code> (path): Numero del tavolo.
Risposte	200 OK: Ritorna gli ordini. 400 Bad Request: Richiesta non valida. 404 Not Found: Tavolo non trovato.

AGGIUNGERE PIZZA ALL'ORDINE DEL TAVOLO

Endpoint	<code>/api/tables/{table_number}/orders</code>
Metodo	POST
Descrizione	Aggiunge un ordine al tavolo specificato.
Richiesta	Parametri: <code>table_number</code> (path): Numero del tavolo. Corpo: Array di <code>pizza_id</code> (number): ID della pizza. <code>quantity</code> (number): Quantità.
Risposte	200 OK: Ordine aggiunto con successo. 400 Bad Request: Richiesta non valida. 404 Not Found: Pizza non trovata.

TIPIZZAZIONE DELLE ENTITÀ

Pizza Response

```
export interface Pizza {  
  id: number  
  name: string  
  description: string  
  ingredients: string  
  price: number  
  image: string  
}
```

Table Order Request Body

```
export interface TableOrderRequestBody {  
  pizza_id: number  
  quantity: number  
}
```

Table Order Response

```
export interface TableOrderResponse {  
  id: number  
  table_number: number  
  pizza_id: number  
  quantity: number  
  status: 'pending' | 'fullfilled'  
  created_at: string  
  updated_at: string  
}
```


DOMANDE A RISPOSTA APERTA

Quesito di area tecnologica 1. Discutere le differenze tra framework e librerie, prendendo come esempio la differenza tra Angular e React, analizzandone gli impatti sul lavoro dello sviluppatore.

Quesito di area tecnologica 2. Descrivere il flusso dei dati tra client e server, infine dettagliare un esempio di scambio di informazioni mediante protocollo REST.

QUESITO 1:

Un framework come Angular impone una struttura precisa e solida, lo sviluppatore deve seguire regole e convenzioni stabilite dal framework stesso. In Angular, tutto è integrato (routing, form, HTTP...), ed è il framework che "richiede" il codice dell'utente. Angular dunque facilita l'organizzazione in progetti complessi ma può risultare più rigido.

Una libreria come React, invece, è più flessibile e friendly, fornisce strumenti per costruire interfacce, ma lascia allo sviluppatore la libertà di decidere come organizzare l'applicazione, quali usare per routing, gestione stato. React dunque richiede più scelte da parte dello sviluppatore, ma offre maggiore libertà e modularità.

QUESITO 2:

Il flusso dei dati tra client e server avviene tramite richieste e risposte HTTP. Il client (browser) invia una richiesta a un server che la elabora e restituisce una risposta, spesso in formato JSON.

Con il protocollo REST, ogni risorsa (es. utenti, prodotti) è accessibile tramite URL. Le operazioni si basano sui verbi HTTP:

- GET per leggere dati
- POST per crearli
- PUT o PATCH per aggiornarli
- DELETE per eliminarli

ESEMPIO API REST:

il client invia una GET /api/prodotti/1

il server risponde con i dttagli del prodotto con quell'ID 1 in formato JSON

MODALITA DI GESTIONE DELLA PROVA E CONSEGNA

SVOLGIMENTO

Accedere alla macchina assegnata con le credenziali che saranno fornite all'inizio della prova.

Le credenziali avranno per la durata della prova in questione tutti i diritti "Amministrativi", così da lasciare al candidato la possibilità di installazione e gestione autonoma di tutte le componenti e funzionalità della propria macchina in modo da permettere la costruzione degli elaborati richiesti nella prova.

Una volta eseguito l'accesso alla macchina, creare sul Desktop una nuova cartella denominata secondo questa nomenclatura "WDV_Esame-2025_xx_yyy_zzzz", dove sostituire i campi non definiti (xx-yyy-zzzz) con i valori qui elencati:

"xx" = Numero di Registro

"yyy" = Cognome

"zzzz" = Nome

Una volta applicate le modifiche al nome della cartella, al suo interno si dovranno salvare tutti i file (progetti, mockup, sviluppo, librerie, template, ecc.) oggetto della prova *ed il file di testo contenete le risposte alle due domande a risposta aperta*

AL TERMINE DELL'ELABORATO

Al termine dell'elaborato, il candidato dovrà inserire all'interno della cartella precedentemente costituita tutti i file sviluppati per la consegna della prova, *il file di testo contenete le risposte alle due domande aperte (possibilmente in formato pdf)* e creare con tutti questi un archivio compresso .zip denominato "WDV_Esame-2025_xx_yyy_zzzz".

Successivamente, il candidato dovrà fare l'accesso alla piattaforma FAD dell'ITS (<https://fad.its-ictpiemonte.it/>) e seguire le seguenti istruzioni per la consegna:

- entrare all'interno del proprio corso "WEB Developer"
- entrare nella cartella "Esame di Stato"
- caricare il file .zip (dimensione massima: 495,00 MB) nella consegna indicata "Prova Teorico Pratica - Consegna archivio".